

Fair and Efficient Task Allocation and Path Planning for Multi-Agent Systems in Disaster Response Using Linear Programming

Trung Hieu Pham
Division of Science, Engineering, Math
Cypress College
Cypress, USA
01971259@cypresscollege.edu

Adam Thorpe
Oden Institute
University of Texas at Austin
Austin, USA
adam.thorpe@austin.utexas.edu

Ufuk Topcu
Department of Aerospace Engineering
and Engineering Mechanics
University of Texas at Austin
Austin, USA
utopcu@utexas.edu

Abstract—This paper presents a linear programming (LP) framework for optimizing task allocation and pathfinding in multi-robot systems, with a specific focus on fairness and efficiency in disaster response scenarios. Applying the Fairness-Enhanced Multi-Agent Deep Deterministic Policy Gradient (FE-MADDPG) and Conflict-Based Search (CBS), the research demonstrates significant improvements in task distribution and path optimization. This contributes to more effective and equitable disaster management solutions.

Keywords— *disaster response, multi-agent systems, robotics, linear programming, FE-MADDPG, CBS*

I. INTRODUCTION

As disaster frequency and severity increase globally, efficient, and fair multi-robot systems become important for enhancing response capabilities in scenarios such as search and rescue, fire suppression, and resource delivery [1]. This work introduces a novel application of linear programming to address these challenges, enhancing both operational efficiency and fairness in robotic task distribution.

II. PRELIMINARIES

A. Problem Statement

In multi-robot systems, balancing optimization and fairness is an important challenge, particularly in disaster response scenarios. Traditional optimization approaches focus on minimizing task completion times and maximizing efficiency, often neglecting fairness in workload distribution among robots [2]. This imbalance can lead to inefficiencies and potential system bottlenecks. Moreover, equitable service to different people is important for public acceptance and operational success. This research aims to address this gap by integrating fairness into task allocation and path planning processes using LP.

B. Related Work

Previous work has primarily focused on optimizing task allocation and pathfinding using various algorithms. However, the aspect of fairness has often been overlooked [2]. Traditional methods, such as greedy algorithms and optimization-based approaches, prioritize efficiency over equitable task distribution. Recent advancements in reinforcement learning, particularly in multi-agent systems, have opened new avenues for integrating fairness into these processes. The use of LP and FE-MADDPG offers a novel solution by balancing efficiency with fairness, addressing the limitations of prior approaches.

III. APPROACH

A. Formulating the problem

We set up a 2D grid world to evaluate the Linear Programming (LP) method, including walls, robots, and fire tasks. We model the problem of allocating tasks and finding paths for multiple robots as a Markov Decision Process (MDP) [3]. MDP consists of five tuples (S, A, R, π , γ). Here's how the elements of the MDP are defined:

- State (S): Represents the grid's current state, including all agents' positions and task statuses, along with fire locations and obstacles.
- Action (A): Derived from LP solutions, these are the movement choices for agents to optimize paths and avoid conflicts.
- Reward (R): Encourages efficient, conflict-free path completion and fair task distribution among agents, penalizing longer paths and conflicts.
- Policy (π): Strategically dictates agents' actions based on LP outputs to achieve optimal and fair paths.
- Discount Factor (γ): Emphasizes the importance of future rewards, essential for considering long-term benefits in path planning.

B. Fairness in Task Assignment

1) Fairness-Enhanced MADDPG

The FE-MADDPG employed in this study extends the conventional MADDPG framework to improve fairness in task allocation among agents. This update is based upon methodologies discussed in [4], adapting their proposed fairness reward function to our application in disaster response robotics. FE-MADDPG creates a more even distribution of tasks.

2) Fair Reward Function

FE-MADDPG adjusts agents' policies based on their relative task loads, using a fairness-oriented reward function [4]:

$$r_t^i = \frac{\varepsilon + |e_t^i / \bar{e}_t - 1|}{\bar{e}_t}$$

r_t^i : The reward for agent i at time t .

ε : A constant to prevent division by zero and ensure numerical stability.

e_t^i : The performance of agent related to distance the agent and the target point.

$\bar{e}_t$: The average performance of n agents at time t .

3) Algorithm Design

The LP formulation aims to minimize the variance in task loads between agents, ensuring that all agents are equally dispatched, which improves overall system efficiency and fairness. This approach uses the following model:

Objective Function:

Minimize the maximum discrepancy in task load among agents.

$$\min \max_i \left| x_i - \frac{1}{n} \sum_{j=1}^n x_j \right|$$

Where x_i represents the task load of agent i , and n is the total number of agents.

Constraints:

1. Each task must be assigned to exactly one agent.
2. Each agent must have the workload distributed as evenly as possible.

Algorithm: FE – MADDPG with LP for Task Allocation

Input: Set of tasks T , Set of agents A

Output: Task assignments for each agent

1. Initialize agent states and policy networks.
2. **For** each episode:
 - a. **For** each agent in A :
 - i. Observe current state s .
 - ii. Select action a using the policy derived from MADDPG adjusted by LP.
 - iii. Execute action a and observe new state s and reward r .
 - b. Update policies using:

- Observed rewards r , states s , and actions a .
- LP to adjust task assignments based on minimizing the discrepancy in task loads.
- c. Train using samples from the replay buffer to minimize the loss function of the MADDPG.
3. Return the final task assignments.

4) Performance evaluation

In this section, we compare the performance of the system before and after using FE-MADDPG.

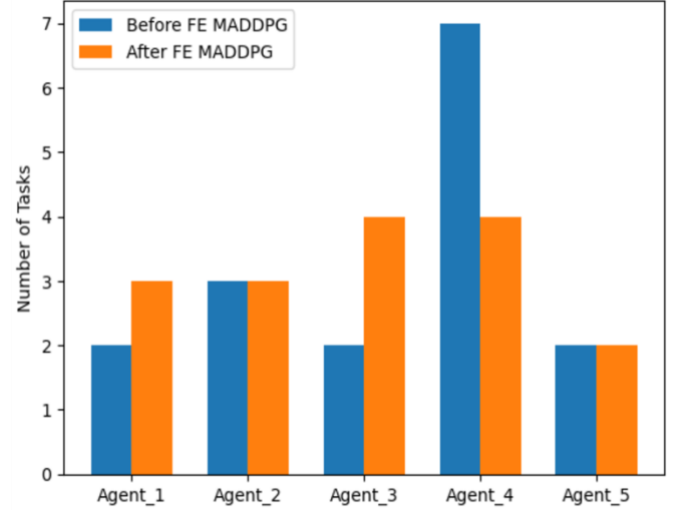


Fig. 1: Task Allocation Comparison

The use of FE-MADDPG resulted in a more even allocation of tasks, as evidenced by a more uniform distribution across agents after applying FE-MADDPG

C. PATH FINDING

We use the Conflict-Based Search (CBS) pathfinding algorithm, guided by previously assigned tasks. CBS aims to find the most optimal paths for agents so that they reach their targets without collisions [5]. This algorithm operates on two levels: the higher level manages conflict resolution through a constraint tree, while the lower level ensures that paths follow these constraints, creating an optimal and conflict-free navigation.

D. LP Integration with CBS

Integrating LP into CBS for path finding involves formulating an optimization problem that not only seeks to minimize the path cost but also ensures compliance with the dynamically changing constraints as conflicts are identified and resolved.

Objective Function:

$$\text{Minimize } \sum_{i=1}^n \sum_{j=1}^m c_{ij} x_{ij}$$

Where c_{ij} is the cost associated with robot i performing task j , and x_{ij} is a binary variable indicating whether path i is

selected (1) or not (0). n is the number of tasks, and m is the number of robots.

Constraints Functions:

- **Collision Avoidance:** Ensure that no two paths share the same grid space at the same time.
 - $X_i + X_j \leq 1$ if paths i and j intersect.
- **Task Completion:** Each task must be completed by exactly one agent.
 - $\sum X_i(t) = 1$ for each task t
- **Feasibility:** Path decisions (X_i) must be binary.
 - $X_i \in \{0, 1\}$

These constraints are dynamically updated as the CBS tree evolves, with new constraints added whenever a conflict is detected between any two agents' paths.

Algorithm: CBS for Pathfinding

Input: Initial paths for all agents
Output: Collision-free paths
1: Initialize the open list with root node containing start paths
2: While open list is not empty:
3: Pop node with the lowest cost
4: Check for conflicts
5: If conflict, create constraints and new nodes
6: Push new nodes to open list
7: If no conflicts, return paths

E. Visualization of after applying CBS



Fig. 2: Visualization Agents Navigating to Assigned Tasks

- **Robots:** Different colored triangles represent different robots, each following an unique path to their assigned tasks.
- **Fire Tasks:** Red squares indicate locations where tasks (fire extinguishing) are located.
- **Obstacles:** Black squares represent impassable areas that robots must navigate around.
- **Paths:** Colored lines show the routes taken by each robot from their starting points to their tasks, optimized for quick response and fairness in task distribution.

This visual evaluation confirms that the CBS algorithm, enhanced by LP for cost efficiency, effectively coordinates multiple robots in complex environments, ensuring they perform tasks quickly and without interference from one another.

IV. RESULT

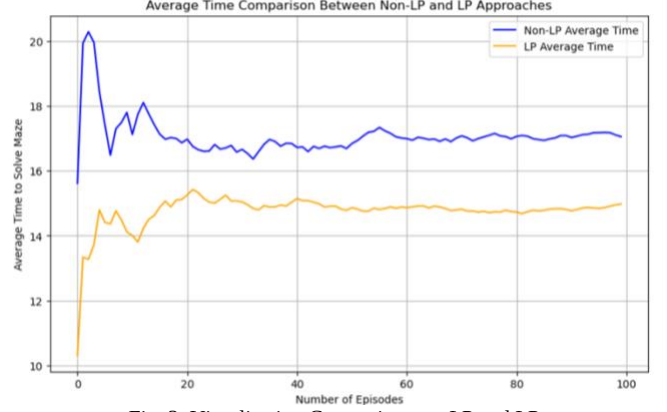


Fig. 3: Visualization Comparing non-LP and LP

From the fig. 3, it is evident that the LP approach consistently outperforms the non-LP approach in terms of average time required to solve the maze across episodes, showing the rapid stabilization and lower variation in solve times which highlights its reliability and efficiency. This efficiency allows robots to complete more tasks in shorter durations, thereby optimizing resource allocation and reducing operational costs.

Consequently, this leads to increased throughput, which is important for scaling operations in disaster environments. The consistent outperformance of the LP approach confirms its potential as a robust tool for improving the effectiveness of multi-agent robotic systems in complex scenarios.

FUTURE WORKS

Future research could explore extending these methods to other areas:

- **Real-Time Data Integration:** By taking real-time data into account, the system can dynamically adjust to changing conditions in disaster environments.
- **Robustness Testing:** Extending testing to include a variety of environmental conditions and disaster scenarios.
- **Scalability Tests:** Expanding the number of agents and tasks in the simulation can provide insights into the scalability of the current LP and FE-MADDPG framework.
- **Application to Other Fields:** Exploring the application of this LP-based approach in other areas such as urban air mobility, agricultural automation, and even large-scale warehouse management.

ACKNOWLEDGMENT

Special thanks to NSF REU Site: Cyberinfrastructure Research for Social Change, award #2150390, and The Texas Advanced Computing Center.

REFERENCES

- [1] Rashmin, G., Oscar Anil, I. E., & James Edward, D. (2023). Global Rapid Post-Disaster Damage Estimation (Grade) Report: February 6, 2023 Kahramanmaraş Earthquakes - Türkiye Report. <https://doi.org/10.1596/39468>.
- [2] Huang, T., Koenig, S., & Dilkina, B. (2021). Learning to resolve conflicts for multi-agent path finding with conflict-based search. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(13), 11246–11253. <https://doi.org/10.1609/aaai.v35i13.17341>.
- [3] Zhang, J., Wang, X., Deng, F., Xin, B., & Chen, W. (2017). Battlefield agent decision-making based on Markov decision process. *Journal of Advanced Computational Intelligence and Intelligent Informatics*, 21(2), 221–227. <https://doi.org/10.20965/jaciii.2017.p0221>.
- [4] Liu, Z., Shi, H., Yan, W., Jin, Z., & Zhou, Y. (2022). A fairness-aware cooperation strategy for multi-agent systems driven by Deep Reinforcement Learning. 2022 41st Chinese Control Conference (CCC). <https://doi.org/10.23919/ccc55666.2022.9902130>.
- [5] Huang, T., Koenig, S., & Dilkina, B. (2021). Learning to resolve conflicts for multi-agent path finding with conflict-based search. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(13), 11246–11253. <https://doi.org/10.1609/aaai.v35i13.17341>.